Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

„Nonprofit adományozó webalkalmazás”

Készítették:

Trefiman Lili Nóra,
Ivák Panna,
Berecz Balázs

Ózd, 2026

Tartalomjegyzék

1. Bevezetés **3**

2. Felhasználói dokumentáció **4**
a. Rendszerkövetelmény 4
b. Webalkalmazás indítása 4
c. Webalkalmazás használata 5

3. Fejlesztői dokumentáció **11**
a. Alkalmazott fejlesztői és csoportmunka eszközök 11
b. Adatbázis 11
c. Frontend 11
d. Backend 12

4. Adatbázis **13**
a. Modell leírása (adatmezők – típusok - leírásuk) 13
b. Táblák, kapcsolatok 16
c. E-K diagram 16

5. Frontend **17**
a. Service-ek: 17
b. Routing és navigáció 20
c. Bejelentkezés és regisztráció 21
d. Adományozói felület 23
e. Kosár és rendelés 24

6. Backend **31**
a. Alkalmazás backend részének részletes leírása + képernyőfotók 31

7. Tesztelés **36**
a. A backend sikeres tesztjeinek képernyőfotói (Thunder Client) 36
b. Az backend sikertelen tesztjeinek képernyőfotói (Thunder Client) 38
c. Hibakezelés a frontend oldalon 40

8. Továbbfejlesztési lehetőségek **45**

9. Irodalomjegyzék **46**

1. Bevezetés

A webalkalmazásunk egy olyan nonprofit szervezet online platformja lehet, amely segíteni szeretne azoknak, akiknek a legnagyobb szükségük van támogatásra. Célja, hogy rászoruló emberekhez juttasson el adományokat, így hozzájárulva a mindennapi életük megkönnyítéséhez. Ez az online felület azért készült, hogy megkönnyítse és bárki számára elérhetővé tegye az adományozást. Az oldal segítségével bárki, bárhonnan könnyedén feltöltheti a megunt vagy már nem használt ruháit, amelyeket mások közvetlenül elérhetnek. A civil szervezetek, akik ténylegesen segítenek a rászorulókon, itt kiválaszthatják azokat az adományokat. Így az adományozók biztosak lehetnek abban, hogy a ruhák valóban jó helyre kerülnek, miközben az online rendszer egyszerűvé, gyorssá és átláthatóvá teszi az egész folyamatot.

Napjainkban számos jótékonysági kezdeményezés létezik, azonban kevés olyan egységes online rendszer áll rendelkezésre, amely közvetlenül összeköti az adományozókat és az alapítványokat egy átlátható, könnyen kezelhető felületen keresztül. Sok esetben az adományozás személyes jelenléthez, eseményekhez vagy közvetítő szervezetekhez kötött, ami megnehezíti a folyamatot és csökkenti a részvételi hajlandóságot. A webalkalmazás célja ennek a problémának a megoldása egy digitális platform segítségével.

A téma választása azért esett erre a területre, mert az adományozás és a társadalmi segítségnyújtás kiemelten fontos szerepet tölt be a mindennapi életben. Emellett egy olyan problémára kínál megoldást, amely sok embert érint: sokan szeretnének segíteni, de nem tudják, hogyan tehetnék meg egyszerűen és hatékonyan. A fejlesztés során cél volt egy olyan rendszer létrehozása, amely egyszerre támogatja az adományozókat és az alapítványokat, miközben modern, felhasználóbarát megoldást nyújt.

A projekt GitHub hivatkozása: https://github.com/BBazsi2006/Projekt.git

2. Felhasználói dokumentáció

a. Rendszerkövetelmény

i. Hardveres rendszerkövetelmények

	Ajánlott	Minimális
Processzor (CPU)	Intel(R)_Core(TM)_i5-14600K_CPU_@_5.30GHz	Intel Core i3-6100T Dual-Core 3.2GHz
Operációs rend-szer (OP)	Windows 11	Windows 10
RAM	16 GB Ram	6 GB RAM
Tárhely	20 GB	6 GB
Internet	Széles sávú	Széles sávú
GPU	NVIDIA GeForce GTX 1050	Intel(R) UHD Graphics 620

ii. Szükséges szoftverek a webalkalmazáshoz

- XAMPP 8.1.12 vagy újabb
- Node.js 22.13.0 vagy újabb
- Composer 2.8.4 vagy újabb
- Visual Studio Code 1.96.0 vagy újabb
- Bármilyen friss és stabil webböngésző

b. Webalkalmazás indítása

A webalkalmazás futtatásához először el kell indítani a XAMPP vezérlőpultot, majd el kell indítani az Apache és a MySQL szolgáltatásokat, mivel az adatbázis ezen keresztül működik. Ezt követően a backendet kell elindítani. Ehhez meg kell nyitni a backend mappát a Visual Studio Code-ban, majd a terminálba be kell írni a *php artisan migrate --seed* parancsot. Ez a parancs létrehozza az adatbázis tábláit, és feltölti azokat a szükséges kezdeti adatokkal. Ezután a *php artisan serve* parancs kiadásával elindítható a Laravel szerver, amely a backend működését biztosítja.

Ezután a frontend indítása következik. Ehhez egy új Visual Studio Code ablakban meg kell nyitni a frontend mappát. Amennyiben a szükséges csomagok még nincsenek telepítve, a *npm install* paranccsal telepíthetők. Ezt követően az *ng serve -o* parancs kiadásával elindítható az Angular alkalmazás, amely automatikusan megnyílik a böngészőben. A frontend és a backend együttes futtatásával a webalkalmazás teljes funkcionalitása elérhetővé válik.

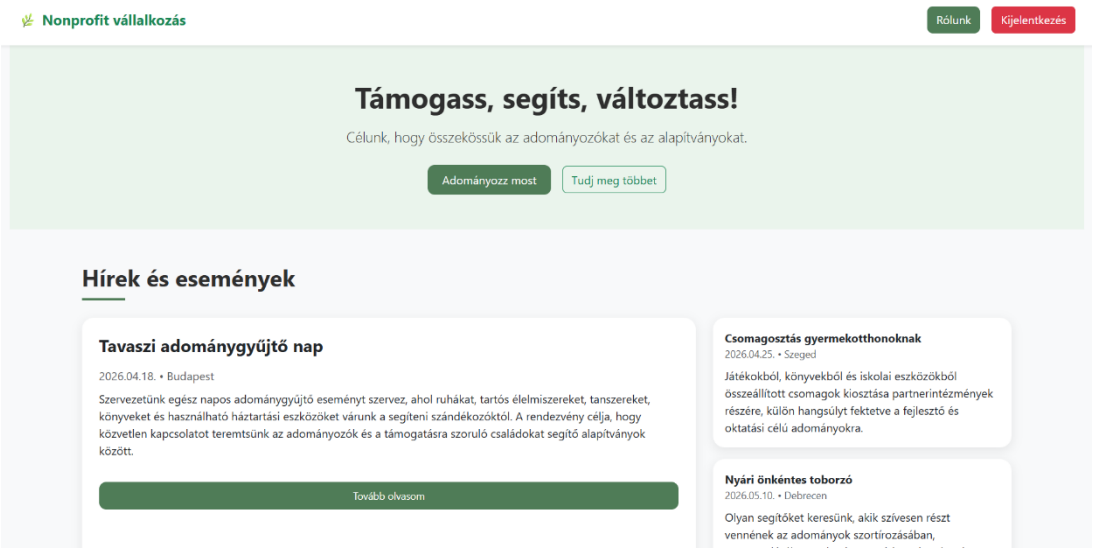
c. Webalkalmazás használata

i. Általános információk a webalkalmazásról

A webalkalmazás célja, hogy összekösse az adományozókat és az alapítványokat, lehetővé téve az adományok egyszerű felajánlását és igénylését. A rendszer háromféle felhasználói szerepkört különböztet meg: adományozót, alapítványt és adminisztrátort. Az adományozók adományokat tölthetnek fel, az alapítványok ezeket megtekinthetik és megrendelhetik, míg az adminisztrátor a rendelések kezeléséért felelős.

Főoldal

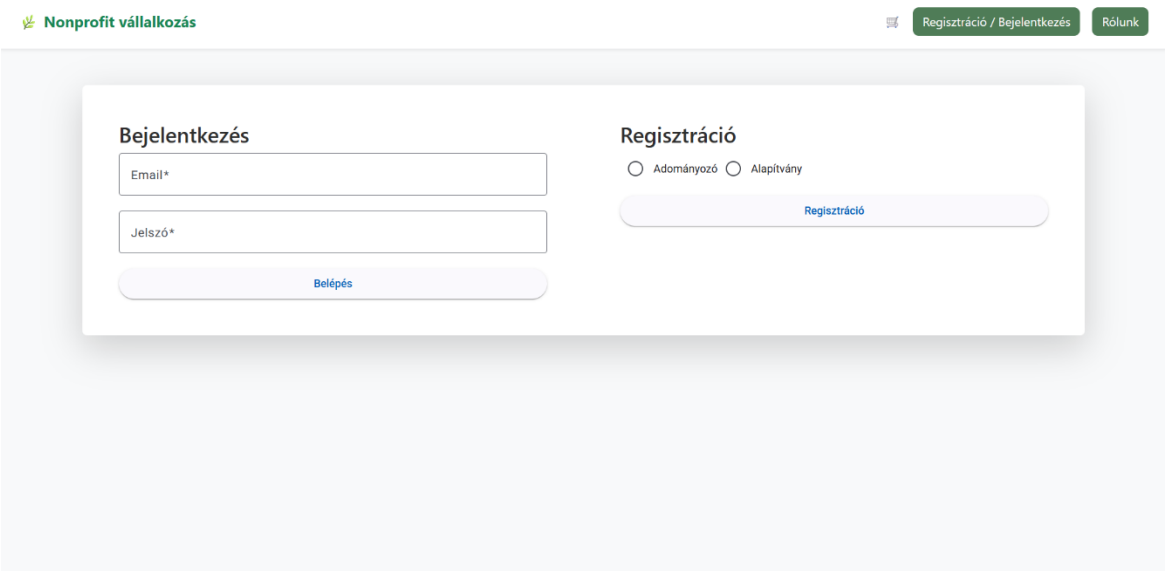
A webalkalmazás megnyitásakor a főoldal jelenik meg. Itt a felhasználó áttekintést kap az oldal céljáról, valamint különböző híreket és eseményeket is megtekinthet. A főoldalon találhatóak még a navigációs elemek, amelyek segítségével elérhető a regisztráció, a bejelentkezés valamint a rólunk oldal.



1. Főoldal

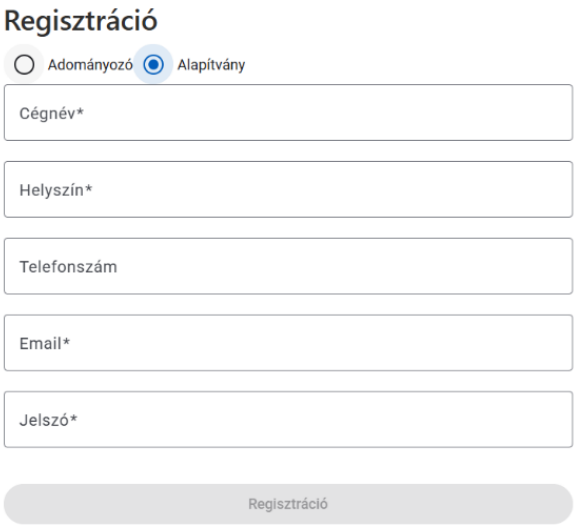
Bejelentkezés és regisztráció

A „Regisztráció / Bejelentkezés” menüpont segítségével a felhasználók új fiókot hozhatnak létre, vagy beléphetnek meglévő fiókjukba. A regisztráció során kiválasztható, hogy a felhasználó adományozóként vagy alapítványként szeretne regisztrálni. Adományozó esetén név, e-mail cím és jelszó megadása szükséges, míg alapítvány esetén cégnév, helyszín, telefonszám, e-mail cím és jelszó megadása történik. Bejelentkezéskor elegendő az e-mail cím és a jelszó megadása.



2. Bejelentkezés és regisztrációs oldal

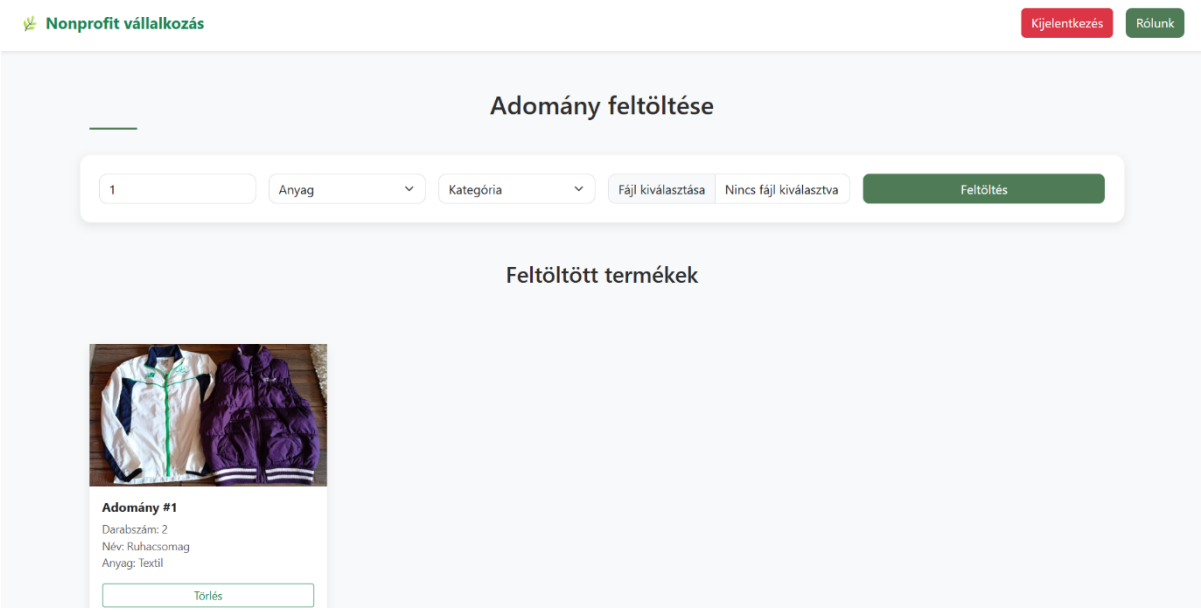
3. Regisztrációs panel



4. Regisztrációs panel

Adományok kezelése (adományozó)

Az adományozó felhasználó bejelentkezés után lehetőséget kap adományok feltöltésére. A feltöltés során megadhatja a darabszámot, kiválaszthatja az anyagot és a kategóriát egy legördülő listából, valamint képet is feltölthet az adott adományhoz. A feltöltött adományok egy listában jelennek meg, ahol a felhasználó kizárólag a saját adományait láthatja és kezelheti.

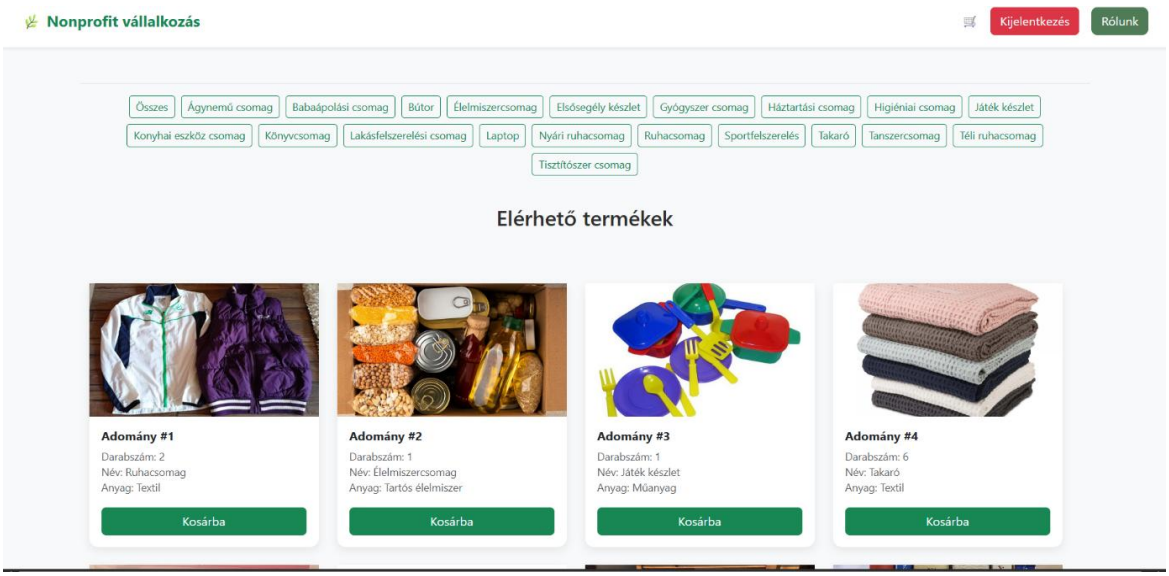


5. Adományozó oldal

6. Anyag legördülő lista

Elérhető adományok (alapítvány)

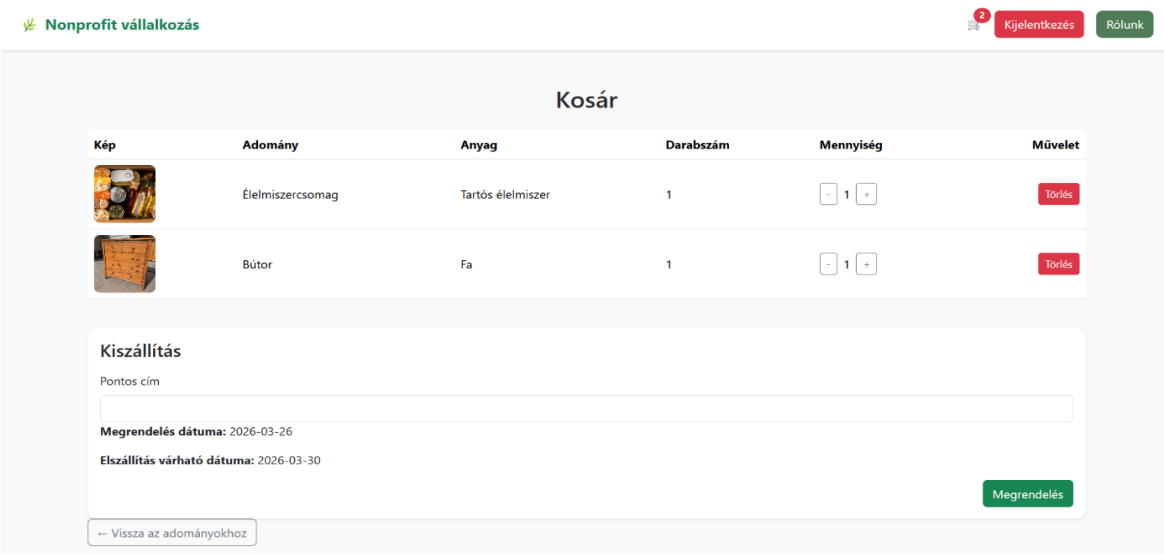
Az alapítványi felhasználó bejelentkezés után megtekintheti az összes elérhető adományt. Az adományok listája kategóriák szerint szűrhető, így könnyebben megtalálhatók a szükséges tételek. Az egyes adományoknál megjelenik a kép, a név, az anyag és a darabszám, valamint lehetőség van azok kosárba helyezésére.



8. Adományok megtekintése alapítványként

Kosár és rendelés

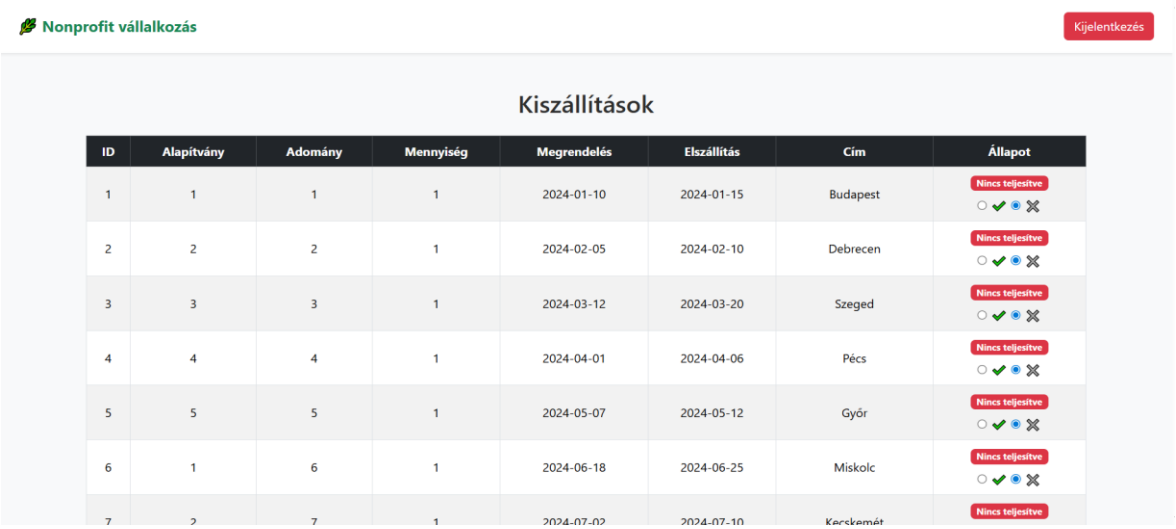
Az alapítvány a kiválasztott adományokat a kosárba helyezheti. A kosár oldalon megjelennek a kiválasztott termékek, azok mennyisége, valamint a hozzájuk tartozó adatok. A rendelés leadásához meg kell adni a pontos címet, valamint a rendszer automatikusan kiszámolja a várható elszállítás dátumát. A rendelés elküldése után a kiválasztott adományok megrendelésre kerülnek.



9. Kosár

Kiszállítások kezelése (admin)

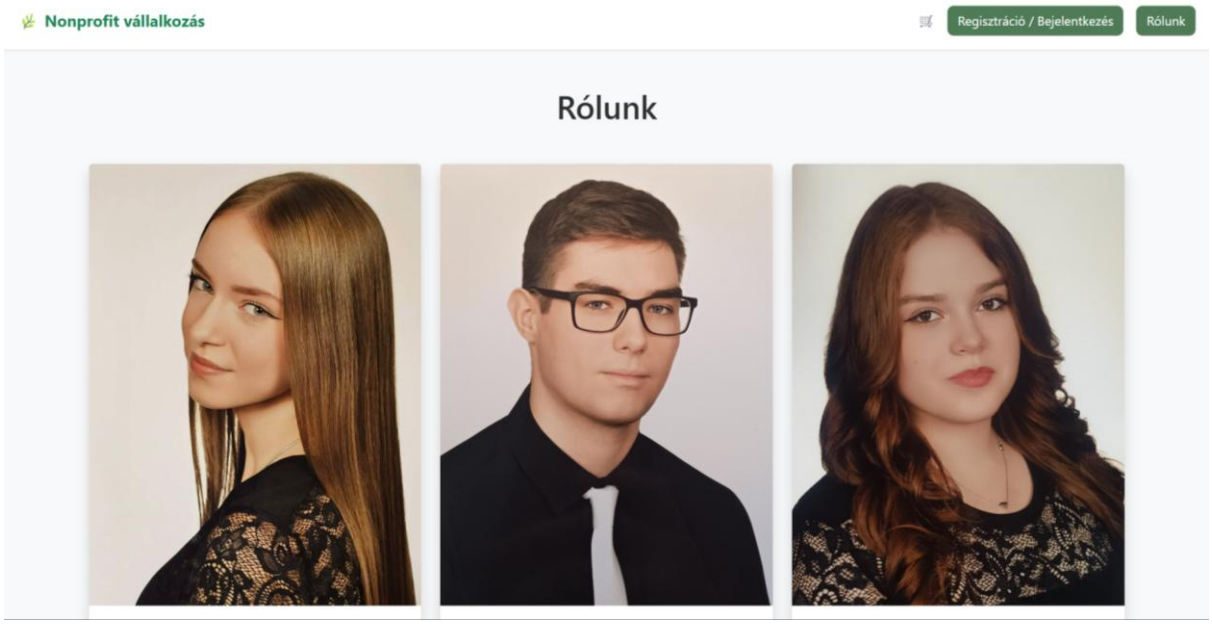
Az adminisztrátor felhasználó jogosultsága kiterjed a rendelések megtekintésére és kezelésére. Az admin felületen egy táblázatban látható az összes rendelés, azok adatai, valamint az aktuális állapotuk. Az adminisztrátor lehetőséget kap a rendelések állapotának módosítására, például azok teljesítettként vagy nem teljesítettként való jelölésére. Emellett meg tudja tekinteni a meglévő készletet.



10. Adminisztrációs oldal, rendelés kezelés

Rólunk oldal

A „Rólunk” oldalon a csapatunk tagjai szerepelnek kártyák formájában, amelyek egy-egy képet és rövid bemutatkozást tartalmaznak, beleértve az iskolai hátterüket és tevékenységeiket. Az oldal alsó részén egy további kártya található, amely az oldal létrejöttének célját ismerteti.



***11**. Rólunk oldal*

3. Fejlesztői dokumentáció

a. Alkalmazott fejlesztői és csoportmunka eszközök

Adatbázis:

- MySQL
- XAMPP
- Visual Studio Code
- Trello
- Figma
- GitHub

b. Adatbázis

A webalkalmazás adatbázisának kezeléséhez a XAMPP környezet került felhasználásra, amely lehetővé teszi a szerver futtatását. A XAMPP tartalmazza az Apache webszervert és a MySQL adatbáziskezelőt, amelyek szükségesek az alkalmazás működéséhez.

Az adatbázis szerkezetének létrehozása a Laravel keretrendszer migrációs rendszerével történik. A *php artisan migrate --seed* parancs futtatásával automatikusan létrejönnek a szükséges táblák, és feltöltésre kerülnek az adatok is.

c. Frontend

A webalkalmazás frontend része Angular keretrendszerrel készült. Ez felel azért, amit a felhasználó lát és használ a böngészőben.

Az alkalmazás több komponensből épül fel, például külön komponens tartozik a főoldalhoz, a bejelentkezéshez, az adományok megjelenítéséhez, a kosárhoz és a kiszállításokhoz. Ezek a komponensek együtt alkotják az alkalmazást.

A felület kialakításához HTML-t, CSS-t, Bootstrap-t és az Angular Material-t használtuk. A Bootstrap segít a reszponzív elrendezés kialakításában, míg az Angular Material előre elkészített, egységes kinézetű komponenseket biztosít (például űrlapokhoz és értesítésekhez).

Az Angular a backenddel HTTP kéréseken keresztül kommunikál, így például az adományok lekérése, a kosár kezelése vagy a rendelések leadása mind a backenddel való kapcsolat segítségével történik.

A felhasználói felület dinamikusan frissül, például amikor egy terméket kosárba helyezünk vagy módosítjuk a mennyiséget. Az Angular segítségével könnyen kezelhető az adatok megjelenítése és a felhasználói események (gombkattintások, űrlapok).

d. Backend

A webalkalmazás backend része Laravel keretrendszerrel készült. Ez felel az adatok kezeléséért és az adatbázissal való kommunikációért.

A backend API-n keresztül kommunikál a frontenddel, vagyis a frontend HTTP kéréseket küld, a backend pedig válaszol az adatokkal. A Laravelben a modellek az adatbázis táblákat írják le (például adományok, alapítványok, kiszállítások), míg a controllerek végzik az adatok feldolgozását.

Az Angular a backenddel HTTP kéréseken keresztül kommunikál, így például az adományok lekérése, a kosár kezelése vagy a rendelések leadása mind a backenddel való kapcsolat segítségével történik.

A felhasználói felület dinamikusan frissül, például amikor egy terméket kosárba helyezünk vagy módosítjuk a mennyiséget. Az Angular segítségével könnyen kezelhető az adatok megjelenítése és a felhasználói események (gombkattintások, űrlapok).

4. Adatbázis

a. Modell leírása (adatmezők – típusok - leírásuk)

Ez az adatbázis egy olyan rendszerhez készült, ahol adományozók különböző alapítványokat támogatnak különféle adományokkal. A cél az volt, hogy egy egyszerű, átlátható és logikus adatmodell készüljön, amiben nyomon lehet követni, hogy ki adományoz, kinek, milyen adományt, és az pontosan mikor kerül kiszállításra.

Az adatbázis összesen 7 táblából épül fel:

1. ember
2. alapitvany
3. adomanyozo
4. adomanynev
5. anyag
6. adomany
7. kiszallitas

<u>1. emberek tábla</u>

Ez a tábla tulajdonképpen egy „közös szülő” tábla, ahol minden szereplő (alapítványok és adományozók) külön kategóriát kap. Így egy helyen nyilvántarthatók a rendszer felhasználói.

Mező neve	**Típus**	**Leírás**
id	int	Az ember egyedi azonosítója. Ez az elsődleges kulcs (PK).
kategoria	varchar(20)	Megmondja, hogy az adott ember „alapitvany” vagy „adomanyozo”.

<u>2. alapitvanyok tábla</u>

Itt vannak tárolva azok az alapítványok, akik fogadják az adományokat. Minden alapítvány hozzá van kötve egy ember rekordhoz.

Mező neve	**Típus**	**Leírás**
alapitvanyId	ind	Az alapítvány egyedi azonosítója. Ez az elsődleges kulcs.
cegnev	varchar(50)	Az alapítvány neve.
helyszin	varchar(50)	Város vagy település, ahol az alapítvány működik.

telszam	int	Telefonszám számként tárolva.
email	varchar(50)	Kapcsolattartási e-mail cím.
jelszo	varchar(12)	A bejelentkezéshez használható jelszó.
emberId	int	Hivatkozás az ember tábla id mezőjére.

3. adomanyozok tábla

Ebben tároljuk az adományozókat: nevüket, elérhetőségüket és a jelszavukat.

Mező neve	**Típus**	**Leírás**
adomanyozoId	int	Az adományozó egyedi azonosítója. Elsődleges kulcs.
nev	varchar(50)	Az adományozó teljes neve.
email	varchar(50)	Kapcsolattartási cím.
jelszo	varchar(50)	A bejelentkezéshez használható jelszó.
emberId	int	Az ember tábla megfelelő rekordjára mutat.

4. adomanynevek tábla

Itt vannak felsorolva a különböző adományfajták (pl. élelmiszer, takaró stb.).

Mező neve	**Típus**	**Leírás**
adomanynevId	int	Az adomány típusának egyedi azonosítója. Elsődleges kulcs.
nev	varchar(50)	Az adomány neve / típusa.

5. anyagok tábla

Ez a tábla azt mondja meg, hogy egy adott adomány milyen anyagból készült (például: fa, textil, műanyag stb.).

Mező neve	**Típus**	**Leírás**
anyagId	int	Anyag típus egyedi azonosítója. Elsődleges kulcs.
anyagnev	varchar(50)	Anyag megnevezése.

6. adomanyok tábla

Itt tároljuk a konkrét adományokat: mit, mennyit és milyen anyagból.

Mező neve	Típus	Leírás
adomanyId	int	Egyedi azonosító. Elsődleges kulcs.
darabszam	int	Hány darab az adott adományból.
nevId	int	Hivatkozás az adomanynev tábla egyik adománytípusára.
adomanyanyagId	int	Hivatkozás az anyag tábla egyik anyagára.
adomanyozoId	int	Hivatkozás arra, hogy ki töltötte fel az adományt.
kep	varchar(255)	Feltöltött vagy megadott kép az adományról.

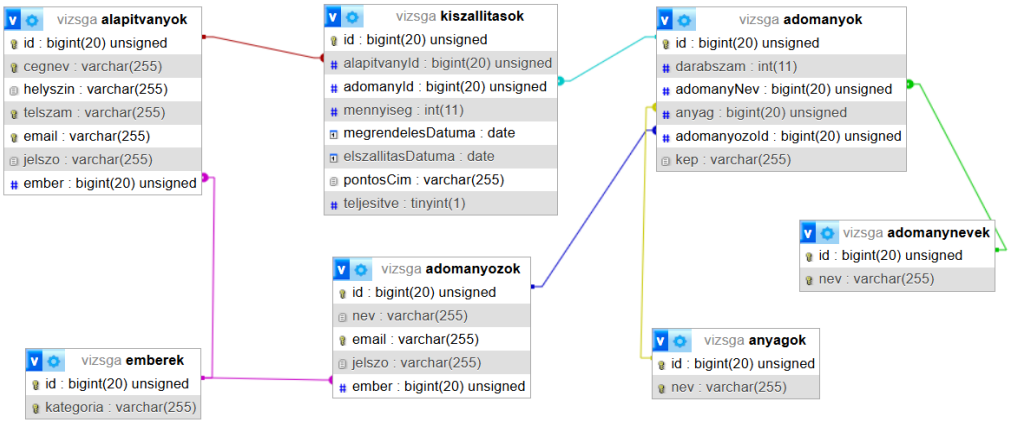
7. <u>kiszallitasok tábla</u>

Ez tárolja a kiszállítási „feladatokat”: melyik alapítvány rendelt, melyik adományról van szó, és mik a dátumok.

Mező neve	Típus	Leírás
munkaId	int	A munka azonosítója. Elsődleges kulcs.
alapitvanyId	int	Annak az alapítványnak az ID-je, aki kapja az adományt.
adomanyId	int	A kiszállított adomány ID-je.
megrendelesDatuma	date	Mikor adták le a rendelést.
elszallitasDatuma	date	Mikor vitték el az adományt.

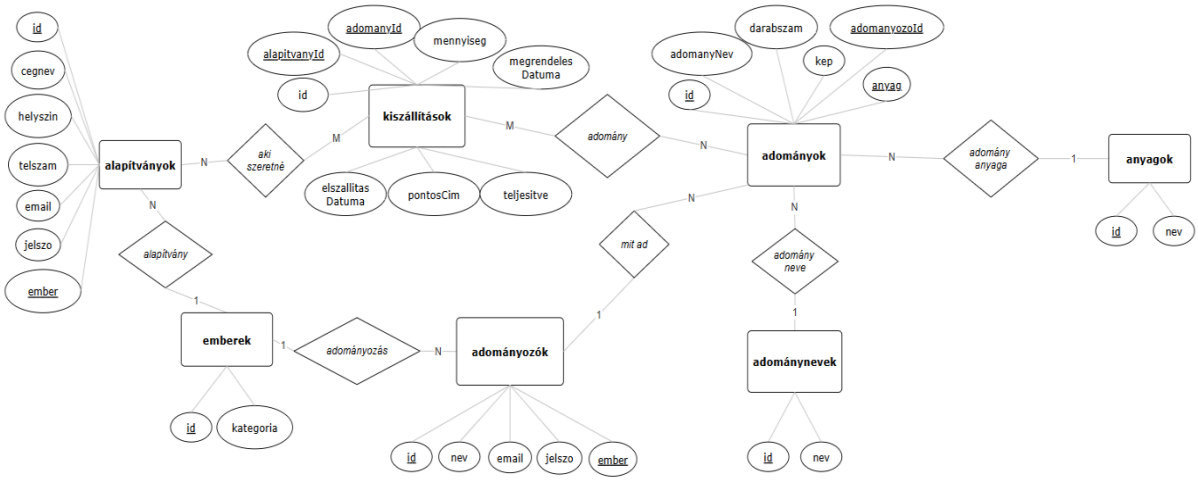
Ez az adatbázis egy adományos rendszerhez készült, ahol követni lehet, hogy ki adományoz, melyik alapítvány kapja, mit kapnak, és mikor szállítják ki. Az *ember* tábla csak arra van, hogy el lehessen dönteni, ki adományozó és ki alapítvány. Az adományoknál külön van kezelve a típus és az anyag, hogy ne legyen minden összekeverve. A *munka* tábla köti össze az alapítványokat a konkrét adományokkal és a dátumokkal. A lényege az, hogy az adatok átláthatók legyenek és vissza lehessen keresni, mi hova került.

b. Táblák, kapcsolatok



11. Táblák

c. E-K diagram



12. EK-model

5. Frontend

A webalkalmazás frontend része Angular keretrendszerrel készült. A felhasználói felület kialakításához HTML és CSS került felhasználásra, amelyet Bootstrap és Angular Material elemek egészítenek ki. Az Angular komponens alapú felépítése lehetővé tette, hogy az alkalmazás különálló, jól átlátható részekből épüljön fel, így a fejlesztés és a későbbi módosítások is egyszerűen megvalósíthatóak voltak.

A frontend feladata, hogy a felhasználó számára biztosítsa az alkalmazás használatához szükséges felületet, valamint kapcsolatot tartson a backenddel. A rendszer különböző komponensekből és service osztályokból épül fel. A komponensek az egyes oldalak és felületek megjelenítéséért felelnek, míg a service-ek az adatkezelést és a backend API-val való kommunikációt végzik.

A projekt frontend része összesen **8 komponensből** és **9 service-ből** áll: navigációs komponens, a főoldal, a bejelentkezési és regisztrációs felület, az adományozói felület, az alapítványi felület, a kosár oldal, valamint az adminisztrációs felület. A felhasználó az Angular Router segítségével tud az egyes oldalak között váltani.

a. Service-ek:

A frontend működésének egyik legfontosabb elemei a service osztályok. Ezek felelősek az adatok kezeléséért, valamint a backenddel való kommunikációért. A service-ek segítségével a komponenseknek nem kell közvetlenül HTTP kéréseket kezelniük.

A service-ek az Angular *HttpClient* segítségével kommunikálnak a backend API-val. A legtöbb service tartalmaz alapvető műveleteket, mint például:

- adatok lekérdezése (GET)
- új adatok létrehozása (POST)
- adatok törlése (DELETE)
- adatok frissítése (UPDATE)

```
getAll(): Observable<Adomany[]> {
  return this.http.get<Adomany[]>(this.apiUrl);
}
```

13. getAll() metódus

Ez a metódus lekéri az összes adományt a backendből, amelyeket a komponensek megjelenítenek a felhasználói felületen.

Új adomány létrehozása esetén:

```
create(adat: FormData): Observable<Adomany> {
  return this.http.post<Adomany>(this.apiUrl, adat);
}
```

14. create() metódus

Ez a metódus elküldi a felhasználó által megadott adatokat a backendnek, amely eltárolja azokat az adatbázisban.

```
delete(id: number): Observable<any> {
  return this.http.delete(`${this.apiUrl}/${id}`);
}
```

15. delete() metódus

A delete() metódus egy HTTP DELETE kérést küld a backend felé, amely a megadott azonosító alapján törli az adott rekordot az adatbázisból.

```
update(id: number, adomany: Adomany): Observable<Adomany> {
  return this.http.put<Adomany>(`${this.apiUrl}/${id}`, adomany);
}
```

16. update() metódus

Az update() metódus segítségével a megadott azonosító alapján frissíteni lehet az adatokat az adatbázisban.

További metódusok:

```
regisztralAdomanyozo(adat: any) {
  return this.http.post(`${this.apiUrl}/adomanyozo`, adat);
}

regisztralAlapitvany(adat: any) {
  return this.http.post(`${this.apiUrl}/alapitvany`, adat);
}
```

17. regisztráló metódusok

A *regisztralAdomanyozo()* és a *regisztralAlapitvany()* metódusok a felhasználók regisztrációját kezelik a frontend oldalon. Mindkét metódus HTTP POST kérést küld a backend API felé, amely az elküldött adatok alapján létrehozza az új felhasználót az adatbázisban.

A két metódus külön végpontot használ, mivel az alkalmazás két különböző felhasználói típust kezel: adományozókat és alapítványokat. Ez lehetővé teszi, hogy a rendszer eltérő jogosultságokkal kezelje a különböző felhasználókat.

```
addToCart(termek: any) {
  const aktualisKosar = this.kosar();
  const index = aktualisKosar.findIndex(t => t.id === termek.id);

  if (index === -1) {
    const ujTermek = {
      ...termek,
      mennyiseg: 1
    };

    this.kosar.set([...aktualisKosar, ujTermek]);
    this.mentes();
    return;
  }
```

18. addToCart() metódus

Az *addToCart()* metódus egy kiválasztott terméket helyez a kosárba. A metódus először ellenőrzi, hogy az adott termék már szerepel-e a kosárban az azonosítója alapján. Ha a termék még

nem található meg a kosárban, akkor a rendszer létrehoz egy új objektumot, amely tartalmazza a termék adatait, valamint egy kezdeti mennyiséget. Ezután a kosár frissítésre kerül, és az adatok mentésre kerülnek.

Ez a megoldás biztosítja, hogy a kosár ne tartalmazzon duplikált elemeket, és minden termékhez megfelelő mennyiség legyen hozzárendelve.

b. Routing és navigáció

Az alkalmazásban az oldalak közötti navigációt az Angular Router biztosítja. Ennek segítségével külön útvonalon érhető el a főoldal, a bejelentkezési oldal, az adományok oldala, a kosár, valamint az adminisztrációs felület.

A navigációs sáv dinamikusan változik a felhasználó állapotától és szerepkörétől függően. Ha a felhasználó nincs bejelentkezve, akkor a rendszer megjeleníti a regisztrációs és bejelentkezési lehetőséget. Bejelentkezés után a menü elemei a felhasználó szerepkörének megfelelően jelennek meg. Az alapítványi felhasználó például látja a kosár ikont, míg az admin felhasználó számára csak a kijelentkezési lehetőség marad elérhető.

A navigáció kezelésére a rendszer feltételeket használ, amelyek a localStorage-ben tárolt felhasználói adatok alapján döntik el, hogy mely elemek jelenjenek meg.

Példa erre az alapítvány szerepkör ellenőrzése:

```
isAlapitvany(): boolean {
  const user = JSON.parse(localStorage.getItem('user') || '{}');
  return user.tipus === 1;
}
```

19. isAlapitvany() függvény

Ez a függvény megvizsgálja a bejelentkezett felhasználó típusát, és eldönti, hogy alapítványról van-e szó.

Főoldal komponens

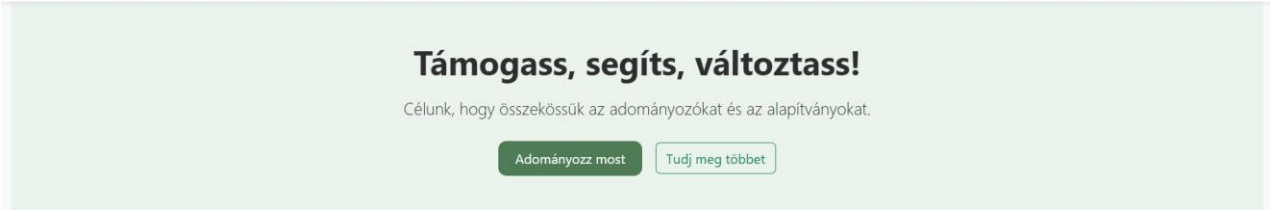
A főoldal az alkalmazás nyitófelülete. Itt a felhasználó rövid bemutatást kap a webalkalmazás céljáról, valamint híreket és eseményeket is megtekinthet. A főoldalon található hero section, amely kiemeli az oldal üzenetét, valamint gombokat biztosít a továbblépéshez.

A főoldal egyik fontos funkciója, hogy a felhasználó szerepkörének megfelelő oldalra navigálja a látogatót az „Adományozz most” gomb segítségével.

```
adomanyozas() {
  const userAdat = localStorage.getItem('user');

  if (userAdat) {
    const user = JSON.parse(userAdat);

    if (user.tipus == 1) {
      this.router.navigate(['/alapitvanyok']);
    } else if (user.tipus == 2) {
      this.router.navigate(['/adomanyok']);
    }
  } else {
    this.router.navigate(['/login']);
  }
}
```

20. Adományozz most - gomb funkció

Ez a metódus ellenőrzi, hogy van-e bejelentkezett felhasználó, és annak típusa alapján a megfelelő oldalra irányítja tovább. Ez biztosítja, hogy az alapítványok és az adományozók külön felületen folytathassák a műveleteket.



21. Hero szekció

c. Bejelentkezés és regisztráció

A bejelentkezési és regisztrációs felület lehetővé teszi új felhasználók létrehozását, valamint meglévő fiókokba való belépést. A rendszer két fő felhasználói szerepkört kezel: adományozót és alapítványt. A regisztráció során a felhasználó kiválaszthatja, hogy melyik típushoz tartozik, és ennek megfelelően jelennek meg a kitöltendő mezők.

A frontend űrlapokkal és validációval biztosítja, hogy a felhasználó csak megfelelő adatokat tudjon elküldeni.

```
belepes(adat: any) {
  return this.http.post(`${this.apiUrl}/belepes`, adat);
}
```

22. belepes() metódus

A service-ben található *belepes()* metódus HTTP POST kérést küld a backend felé a megadott bejelentkezési adatokkal.

A komponens a service *belepes()* metódusát hívja meg, majd a backend válasza alapján feldolgozza a felhasználó adatait. Sikeres bejelentkezés esetén a rendszer eltárolja a felhasználó adatait a localStorage-ben, betölti a kosarat, visszajelzést ad, majd a felhasználó típusától függően a megfelelő oldalra navigál.

```
this.bejelentkezesService.belepes(adat).subscribe({
  next: (valasz: any) => {
    const admin = this.loginEmail === 'admin@gmail.com';

    const user = {
      ...valasz,
      admin: admin
    };


    localStorage.setItem('user', JSON.stringify(user));
    this.kosarService.betoltKosar();

    this.uzenet('Sikeres bejelentkezés!', 'success');

    if (admin) {
      this.router.navigate(['/kiszallitasok']);
    }
    else if (valasz.tipus == 1) {
      this.router.navigate(['/alapitvanyok']);
    }
    else if (valasz.tipus == 2) {
      this.router.navigate(['/adomanyok']);
    }
    else {
      alert("Ismeretlen felhasználótípus");
    }
  },
```

23. Bejelentkezési folyamat kezelése

d. Adományozói felület

Az adományozói felület lehetőséget biztosít arra, hogy a bejelentkezett adományozó új adományokat töltsön fel a rendszerbe, valamint megtekintse a saját korábban feltöltött adományait. A felületen megadható az adomány darabszáma, kiválasztható a megfelelő kategória és anyag, valamint kép is feltölthető az adott termékhez. Az oldal célja, hogy az adományok rögzítése egyszerűen és átlátható módon történjen.

```
const formData = new FormData();
formData.append('darabszam', String(this.ujDarabszam));
formData.append('adomanyNev', String(this.kivalasztottNevId));
formData.append('anyag', String(this.kivalasztottAnyagId));
formData.append('email', user.email);

if (this.selectedFile) {
  formData.append('kep', this.selectedFile);
}

this.adomanyService.create(formData).subscribe({
  next: (valasz: any) => {
    console.log('Sikeres mentés:', valasz);
    this.adomanyok.update(lista => [...lista, valasz]);
```

24. Adomány feltöltése FormData használatával

A feltöltés során a rendszer *FormData* objektumot használ, amely lehetővé teszi, hogy a szöveges adatok és a kiválasztott kép egyetlen kérésben kerüljenek elküldésre a backendnek. A *create()* metódus sikeres lefutása után a frontend frissíti az adományok listáját, így az újonnan feltöltött adomány azonnal megjelenik a felületen.

Alapítványi felület

Az alapítványi felületen a felhasználó megtekintheti az összes elérhető adományt. Az adományok kártyás elrendezésben jelennek meg, képpel, névvel, anyaggal és darabszámmal együtt. A rendszer lehetőséget biztosít az adományok kategória szerinti szűrésére is, így az alapítvány könnyebben megtalálhatja a szükséges termékeket.

Az alapítvány a kiválasztott adományokat kosárba helyezheti.

```
kosarbaRak(adomany: any) {
  this.kosarService.addToCart(adomany);
```

25. Adomány kosárba helyezése

A komponens a *kosarbaRak()* metódus segítségével hívja meg a KosarService *addToCart()* függvényét.

```
addToCart(termek: any) {
  const aktualisKosar = this.kosar();
  const index = aktualisKosar.findIndex(t => t.id === termek.id);

  if (index === -1) {
    const ujTermek = {
      ...termek,
      mennyiseg: 1
    };

    this.kosar.set([...aktualisKosar, ujTermek]);
    this.mentes();
    return;
  }
```

26. addToCart() metódus

Az *addToCart()* metódus egy kiválasztott terméket helyez a kosárba. A rendszer először ellenőrzi, hogy az adott termék már szerepel-e a kosárban. Ha még nem található meg benne, akkor új elemként hozzáadja, és kezdeti mennyiségként 1 darabot állít be. A kosár frissítése után az adatok mentésre kerülnek a localStorage-ben. Ha elfogy a termék, azaz a darabszám 0 lesz, akkor az oldalon a gomb inaktív lesz és a Kosárba felirat helyett az Elfogyott felirat látható.

e. Kosár és rendelés

A kosár feladata az alapítvány által kiválasztott adományok összegyűjtése és kezelése. A felhasználó itt módosíthatja a mennyiséget, törölheti a tételeket, valamint leadhatja a rendelést.

A kosár működését külön service kezeli. A KosarService signal segítségével tárolja a kosár tartalmát, így a felület automatikusan frissül minden változtatás után.

```
const kosarTetelek = this.kosarService.kosar();
let sikeresDb = 0;
let hibasDb = 0;

kosarTetelek.forEach((termek: any) => {
  const adat = {
    alapitvanyId: alapitvanyId,
    adomanyId: termek.id,
    megrendelesDatuma: this.maiDatum,
    elszallitasDatuma: this.elszallitasDatuma,
    pontosCim: this.pontosCim,
    mennyiseg: Number(termek.mennyiseg)
  };
```

27. Rendelési adatok előkészítése

A metódus a kosár tételeit dolgozza fel, és minden egyes termékhez létrehoz egy rendelési adatobjektumot, amely tartalmazza a szükséges adatokat a backend számára.

```
this.kiszallitasService.rendeles(adat).subscribe({
  next: (valasz) => {
    console.log('Sikeres rendelés:', valasz);
    sikeresDb++;

    if (sikeresDb + hibasDb === kosarTetelek.length) {
      if (hibasDb === 0) {
        this.uzenet('A megrendelés sikeresen elküldve!', 'success');
        this.kosarService.clearCart();
        this.elszallitasDatuma = this.szamolElszallitasDatum();
        this.pontosCim = '';
      } else {
        this.uzenet(`${sikeresDb} sikeres, ${hibasDb} hibás rendelés.`, 'error');
      }
    }
  },
  error: (err) => {
    console.log('Rendelési hiba:', err);
    console.log('Backend válasz:', err.error);
    hibasDb++;

    if (sikeresDb + hibasDb === kosarTetelek.length) {
      this.uzenet(`${sikeresDb} sikeres, ${hibasDb} hibás rendelés.`, 'error');
    }
  }
});
});
```

28. Rendelés elküldése

A rendelés elküldése során a rendszer külön kezeli a sikeres és hibás kéréseket.

Miután az összes rendelési kérés feldolgozásra került, a rendszer ellenőrzi az eredményeket, és ennek megfelelően visszajelzést ad a felhasználónak. Sikeres rendelés esetén a kosár kiürítésre kerül, valamint az űrlap mezői visszaállnak alapértelmezett állapotba.

Hiba esetén a felhasználó tájékoztatást kap arról, hogy hány rendelés volt sikeres, és hány végződött hibával.

Admin felület

Az admin felület a kiszállítások és a készlet kezelésére szolgál. Az admin felhasználó itt egy táblázatos felületen megtekintheti az összes rendelést, azok adatait, valamint módosíthatja az állapotukat.

A kiszállítás állapotának módosítását a *teljesitesValtozas()* metódus végzi.

```
teljesitesValtozas(kiszallitas: Kiszallitas, ertek: boolean) {
  this.kiszallitasService.update(kiszallitas.id, {
    teljesitve: ertek
  }).subscribe({
    next: () => {
      this.kiszallitasok.update(lista =>
        lista.map(k =>
          k.id === kiszallitas.id
            ? { ...k, teljesitve: ertek }
            : k
        )
      );
    },
    error: (err) => {
      console.log('Hiba mentés közben:', err);
      alert('Nem sikerült a státusz módosítása.');
    }
  });
}
```

29. teljesitesValtozas() metódus

A *teljesitesValtozas()* metódus lehetővé teszi egy kiszállítás teljesítési állapotának módosítását. A metódus meghívja a kiszallitasService.update() függvényt, amely elküldi a módosított adatot a backend felé.

Az admin felületen ezen kívül külön táblázat jeleníti meg a megmaradt készletet, ahol a rendszer mutatja a termék képét, nevét, mennyiségét és a feltöltő nevét is.

Modellek:

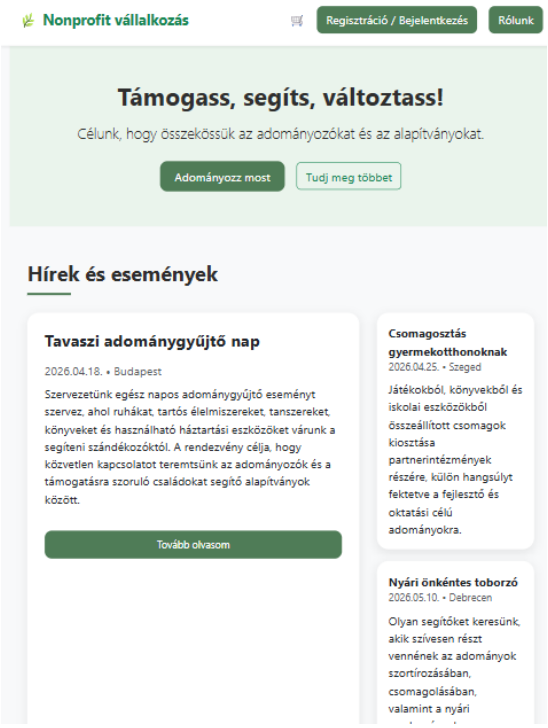
A frontendben modellek is találhatók, amelyek az adatbázisból érkező adatok szerkezetét írják le. Ezek segítenek abban, hogy a TypeScript pontosabban tudja kezelni az adatokat, és csökkentik a hibalehetőségeket.

A modellek például leírják, hogy egy adomány, egy alapítvány vagy egy kiszállítás milyen mezőkből áll. Ez különösen fontos a service-eknél és a komponensekben történő adatkezelés során.

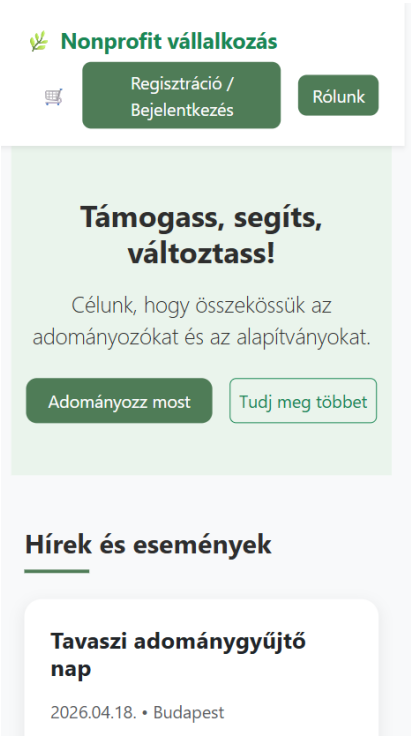
Reszponzivitás:

Az alkalmazás minden felülete reszponzív kialakítással készült, így különböző képernyőméreteken (asztali gép, tablet, mobil) is megfelelően jelenik meg. A Bootstrap rácsrendszerének és saját CSS megoldásoknak köszönhetően az elemek automatikusan átrendeződnek, igazodva a rendelkezésre álló helyhez. A kártyák, űrlapok és navigációs elemek minden eszközön jól használhatóak és átláthatóak maradnak.

Főoldal:

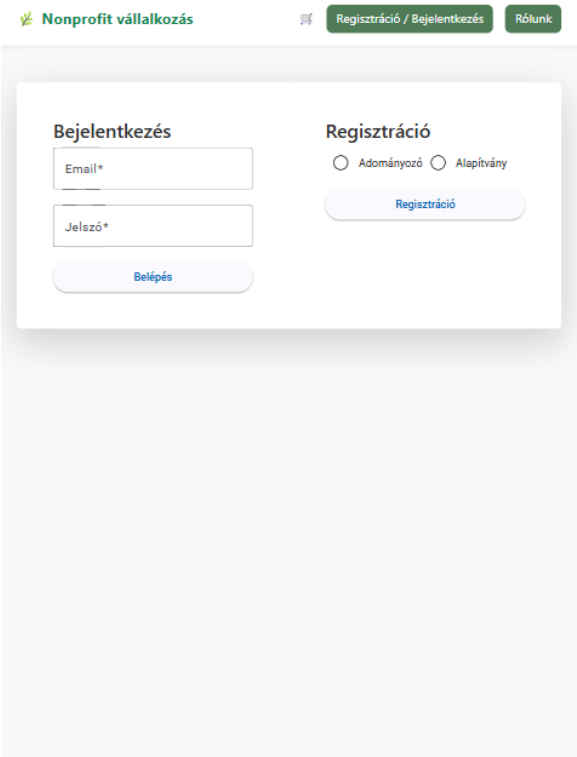


30. Főoldal tablet nézetben

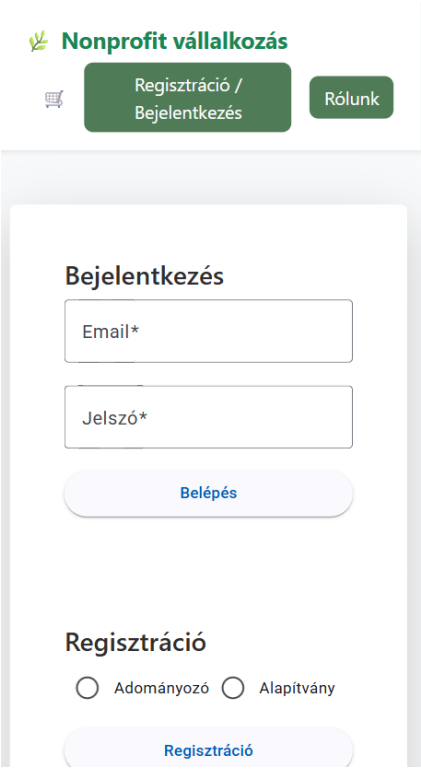


31. Főoldal mobil nézetben

Bejelentkezés/regisztrációs oldal:

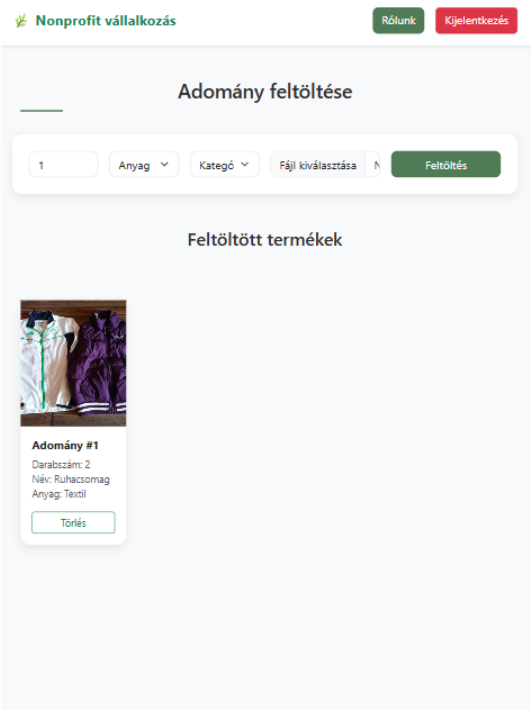


32. Bejelentkezés/regisztrációs oldal tablet nézetben



34. Bejelentkezés/regisztrációs oldal mobil nézetben

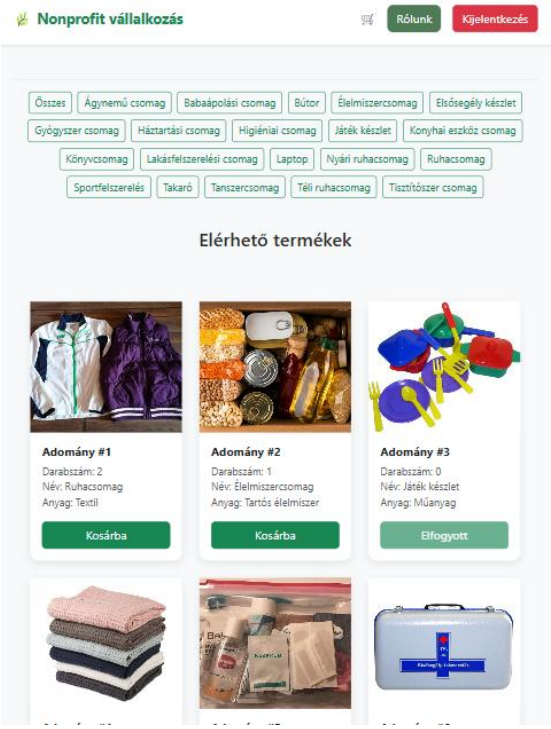
Adományozó oldal:



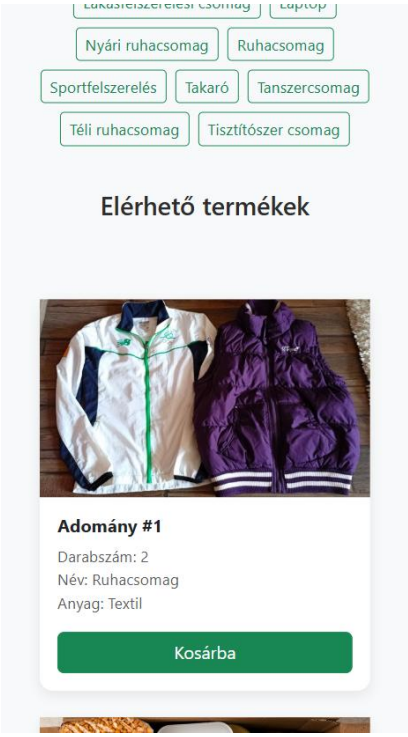
35. Adományozó oldal tablet nézetben

36. Adományozó oldal mobil nézetben

Alapítványok oldal:

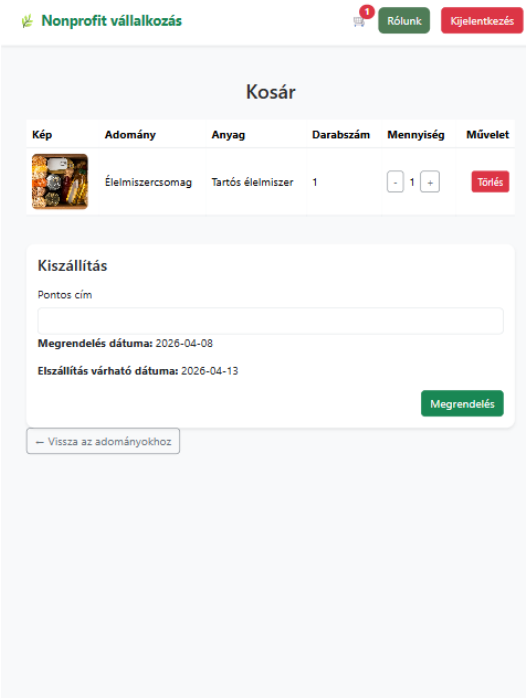


37. Alapítványok oldal tablet nézetben

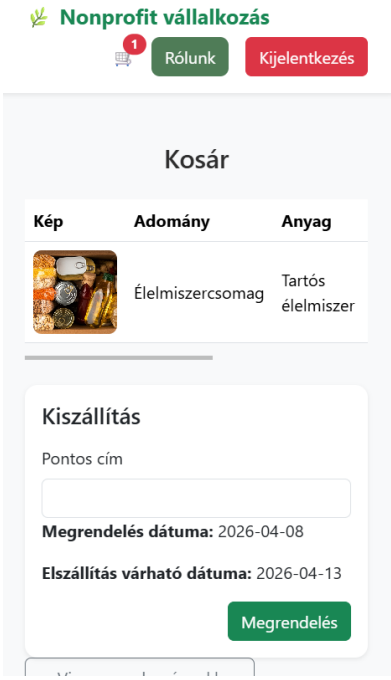


38. Alapítványok oldal mobilnézetben

Kosár:

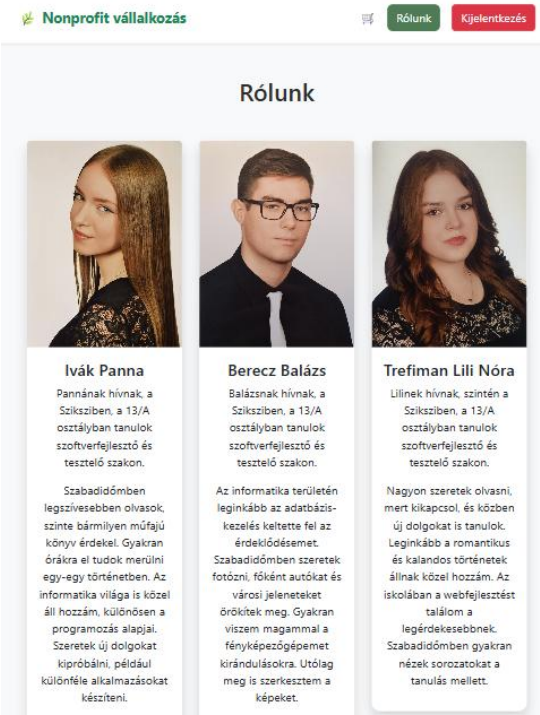


39. Kosár tablet nézetben

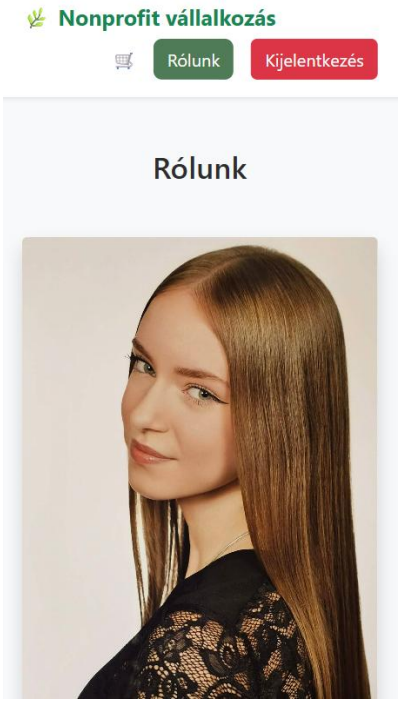


40. Kosár mobil nézetben

Rólunk oldal:

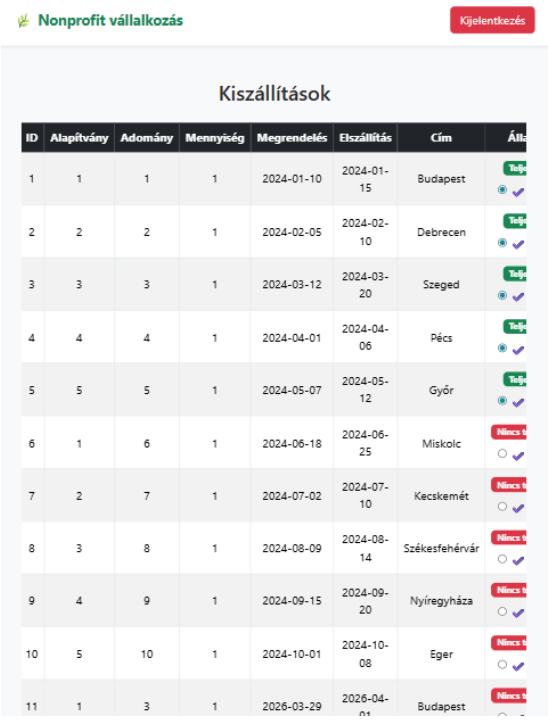


41. Rólunk oldal tablet nézetben

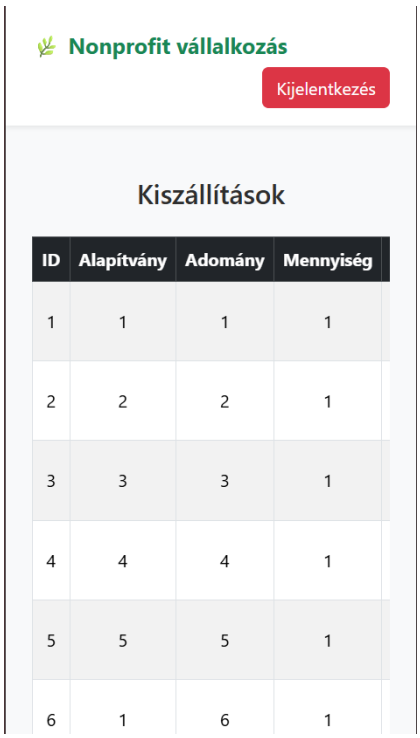


42. Rólunk oldal mobil nézetben

Admin oldal:



43. *Admin oldal tablet nézetben*



44. Admin oldal mobil nézetben

6. Backend

a. Alkalmazás backend részének részletes leírása + képernyőfotók

A backend feladata az adatok kezelése, tárolása és kiszolgálása a frontend felé egy REST API-n keresztül. A rendszer Laravel keretrendszerben készült, és különböző controllereken keresztül valósítja meg az egyes funkciókat: adatok lekérdezése, új adatok létrehozása, meglévő adatok módosítása, adatok törlése. Minden controller metódus felelős egy adott művelet végrehajtásáért, és szabványos HTTP kéréseken (GET, POST, PUT, DELETE) keresztül érhető el. A metódusok feldolgozzák a beérkező kéréseket, majd megfelelő választ adnak vissza JSON formátumban.A válaszok tartalmazzák az adott művelet eredményét, valamint a hozzá tartozó HTTP státuszkódot, amely jelzi a kérés sikerességét vagy sikertelenségét.

i. index():

Az index() metódus az összes alapítvány, illetve adomány adatát lekérdezi az adatbázisból. Ez biztosítja, hogy a frontend egy listában meg tudja jeleníteni az összes alapítványt és adományt a kapcsolódó adatokkal együtt. Ezek a kérések nem módosítják az adatokat, csak visszaadják a kért rekordokat vagy egy adott azonosítóhoz tartozó elemet.

```
public function index()
{
    return Kiszallitas::with(['alapitvany', 'adomany'])->get();
}
```

45. index() metódus

ii. store():

A store() metódus új rekord létrehozását végzi, például új adomány, adományozó vagy kiszállítás rögzítését. A kliens elküldi a szükséges mezőket (alapitvanyId, adomanyId stb.), a backend validálja az adatokat, majd elmenti az adatbázisba. Sikeres létrehozás után a rendszer visszaadja az új erőforrás adatait. A képen a kiszállítás store() metódusa látható.

```php
public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'alapitvanyId' => 'required',
        'adomanyId' => 'required',
        'megrendelesDatuma' => 'required',
        'elszallitasDatuma' => 'required'
    ]);

    if ($validator->fails()) {
        return response()->json(['Hiba' => 'Valamelyik kötelező adat hiányzik'],401);
    }

    $kiszallitas = Kiszallitas::create($request->all());

    return response()->json(['Siker' => 'Kiszállítás rögzítve'],201);
}
```

46. store() metódus

iii. update():

Az update() metódus meglévő adatok frissítésére használható, például egy kiszállítás adatai vagy egy adomány darabszáma módosítható vele. A kérés tartalmazza a frissített mezőket, amelyeket a backend az adott azonosítóhoz tartozó rekordban felülír. Laravelben ez az update() metódusban történik validáció és mentés után.

```php
public function update(Request $request, $id)
{
    $kiszallitas = Kiszallitas::find($id);

    if (is_null($kiszallitas)) {
        return response()->json(['Hiba' => 'Nem létezik kiszállítás ilyen ID-val'],404);
    }

    $kiszallitas->update($request->all());

    return response()->json(['Siker' => 'Kiszállítás frissítve'],200);
}
```

47. update() metódus

iv. destroy():

A destroy() végpont egy adott rekord törlésére szolgál, például egy kiszállítás eltávolítására az adatbázisból. A backend az ID alapján megkeresi a megfelelő sort, majd végrehajtja a törlést,

figyelembe véve a kapcsolódó táblák közti relációkat. A válasz általában egy sikeres státusz vagy visszaigazoló üzenet.

```
public function destroy($id)
{
    $kiszallitas = Kiszallitas::find($id);

    if (is_null($kiszallitas)) {
        return response()->json(['Hiba' => 'Nem létezik kiszállítás ilyen ID-val'],404);
    }

    $kiszallitas->delete();

    return response()->json(['Siker' => 'Kiszállítás törölve'],200);
}
```

48. destroy() metódus

getBiggerNumberOfPieces()

Ez a függvény azokat az adományokat kérdezi le az adatbázisból, amelyek darabszáma nagyobb, mint a paraméterként megadott érték. Ha létezik legalább egy ilyen rekord (exists()), akkor visszaadja a találatokat (get()). Ha nincs ilyen adomány, akkor egy 404-es hibakódú válasz érkezik "nincs ilyen adomány" üzenettel.

```
public function getBiggerNumberOfPieces($db){
    $adomany = Adomany::where('darabszam','>', $db);
    if($adomany->exists())
    {
        return $adomany->get();
    }
    else
    {
        return response("nincs ilyen adomány",404);
    }
}
```

49. getBIggerNumberOfPieces() metódus

getSmallerNumberOfPieces()

Ez a függvény azokat az adományokat kérdezi le az adatbázisból, amelyek darabszáma kisebb, mint a paraméterként megadott érték. Ha létezik legalább egy ilyen rekord (exists()), akkor visszaadja a találatokat (get()). Ha nincs ilyen adomány, akkor egy 404-es hibakódú válasz érkezik "nincs ilyen adomány" üzenettel.

```
public function getSmallerNumberOfPieces($db){
    $adomany = Adomany::where('darabszam','<', $db);
    if($adomany->exists())
    {
        return $adomany->get();
    }
    else
    {
        return response("nincs ilyen adomány",404);
    }
}
```

50. getSmallerNumberOfPieces() metódus

belepes függvény:

Ez a függvény a bejelentkezést kezeli az alkalmazásban. A felhasználó megad egy email címet és jelszót, majd a rendszer ellenőrzi, hogy ezek helyesek-e. Először megnézi, hogy az adatok egy alapítványhoz tartoznak-e. Ha igen, visszaküldi az alapítvány adatait. Ha nem, akkor megnézi, hogy egy adományozóhoz tartoznak-e, és ha igen, annak az adatait adja vissza. Ha egyik esetben sem talál egyezést, akkor hibát jelez, vagyis azt, hogy az email vagy a jelszó nem megfelelő.

```php
public function belepes(Request $request)
{
    $email = $request->email;
    $jelszo = $request->jelszo;

    $alapitvany = Alapitvany::where('email', $email)
        ->where('jelszo', $jelszo)
        ->first();

    if ($alapitvany) {
        return response()->json([
            'tipus' => 1,
            'alapitvanyId' => $alapitvany->id,
            'cegnev' => $alapitvany->cegnev
        ]);
    }

    $adomanyozo = Adomanyozo::where('email', $email)
        ->where('jelszo', $jelszo)
        ->first();

    if ($adomanyozo) {
        return response()->json([
            'tipus' => 2,
            'nev' => $adomanyozo->nev,
            'email' => $adomanyozo->email
        ]);
    }

    return response()->json([
        'hiba' => 'Hibás email vagy jelszó'
    ], 401);
}
```

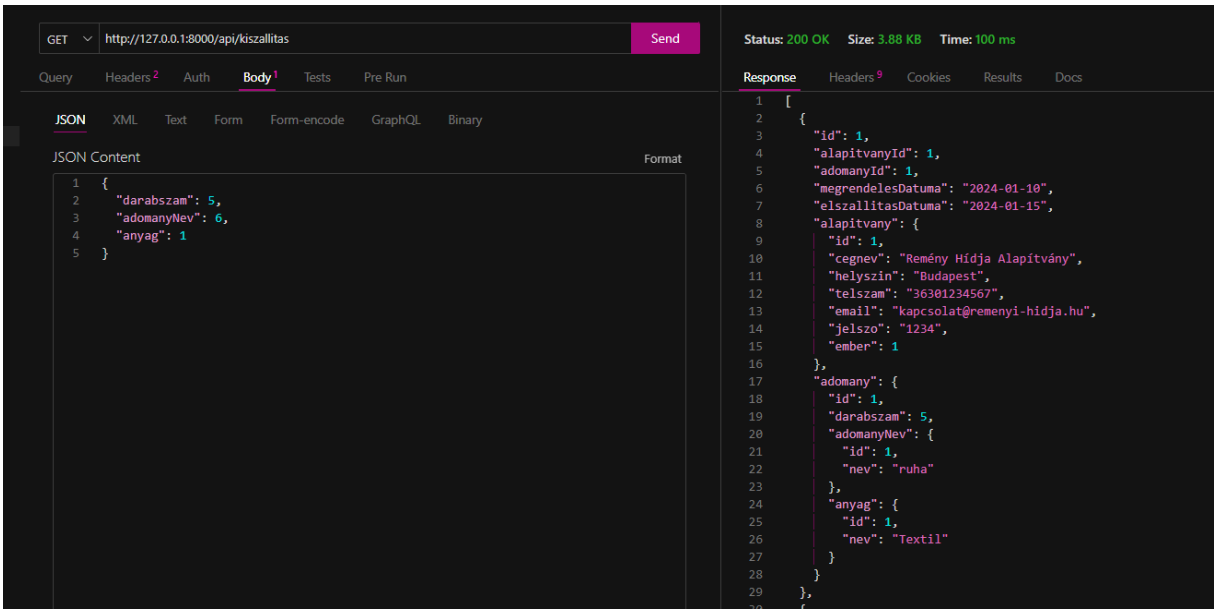
51. belepes() metódus

7. Tesztelés

a. A backend sikeres tesztjeinek képernyőfotói (Thunder Client)

GET:

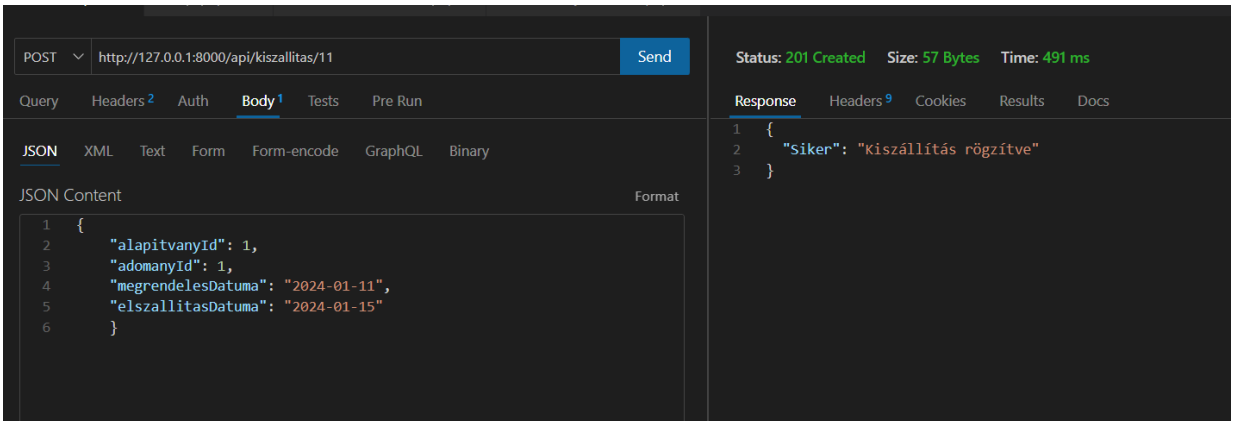
A GET kérések az adatok lekérdezésére szolgálnak a backendből. Ezek segítségével a frontend hozzáférhet az adatbázisban tárolt információkhoz, például az adományokhoz, alapítványokhoz vagy kiszállításokhoz. A válasz JSON formátumban érkezik vissza, amely tartalmazza a szükséges adatokat a megjelenítéshez.



52. Sikeres Get teszt

POST:

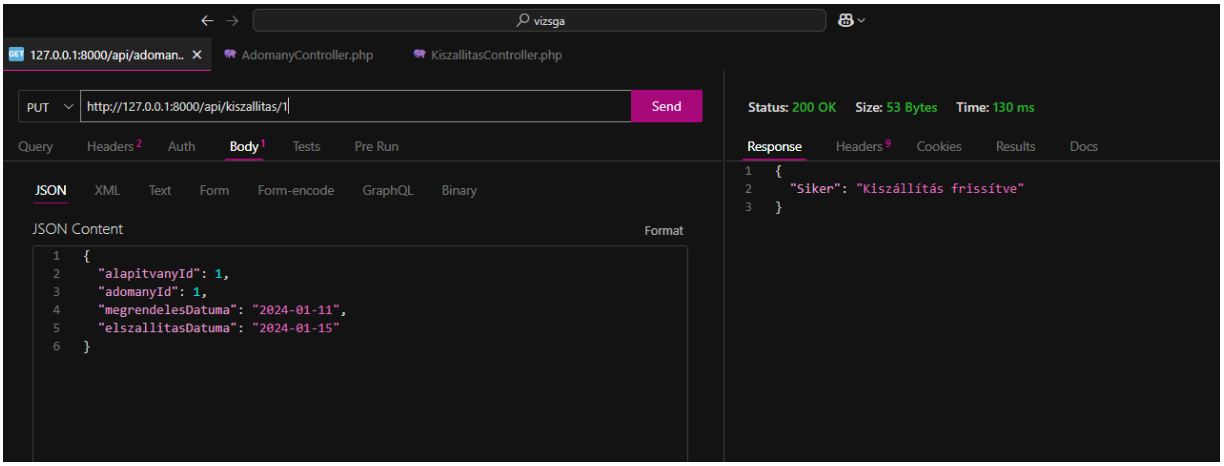
A POST kérések új adatok létrehozására szolgálnak. A frontend ezen keresztül küldi el a felhasználó által megadott adatokat a backendnek, amely ezeket feldolgozza és eltárolja az adatbázisban. A sikeres művelet után a rendszer visszajelzést ad a mentés eredményéről.



53. Sikeres Post teszt

PUT:

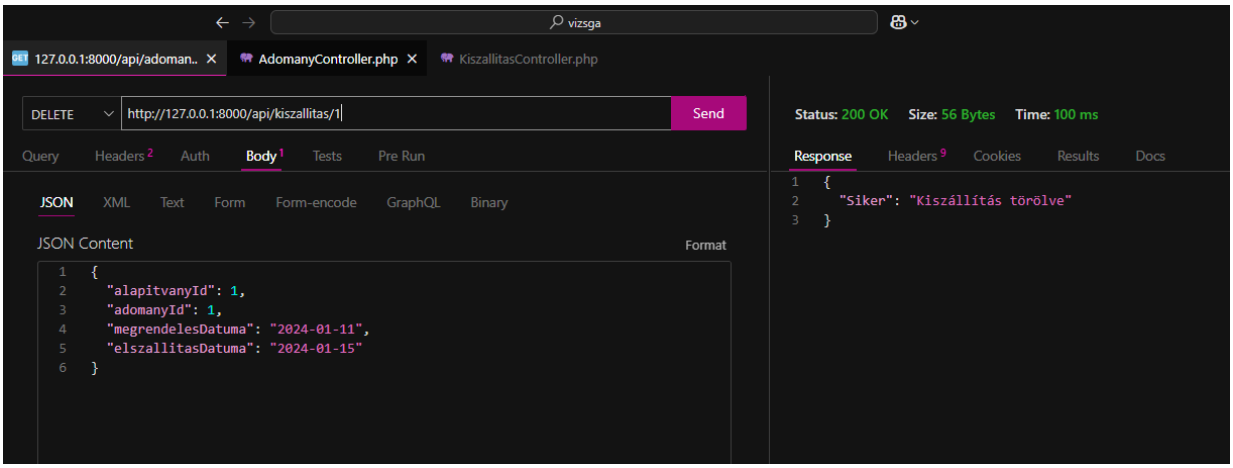
A PUT kérések meglévő adatok módosítására használhatók. A frontend elküldi a módosított adatokat, a backend pedig frissíti az adott rekordot az adatbázisban. Ez például használható állapotok változtatására vagy adatok szerkesztésére.



54. Sikeres Put teszt

DELETE:

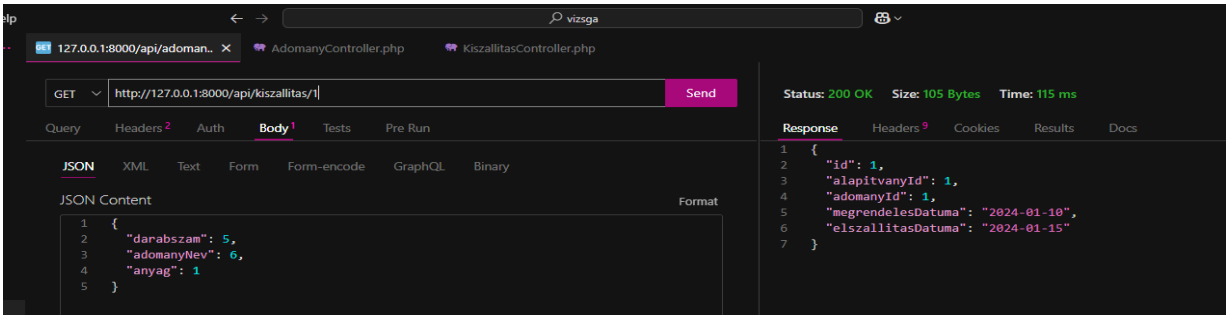
A DELETE kérések az adatok törlésére szolgálnak. A backend a megadott azonosító alapján eltávolítja az adott rekordot az adatbázisból. Ez lehetővé teszi például egy adomány vagy más adat végleges törlését.



55. Sikeres Delete teszt

GETBYID:

A GET BY ID típusú kérés egy konkrét adat lekérdezésére szolgál azonosító alapján. A frontend elküldi az adott elem azonosítóját, a backend pedig visszaadja a hozzá tartozó részletes adatokat.

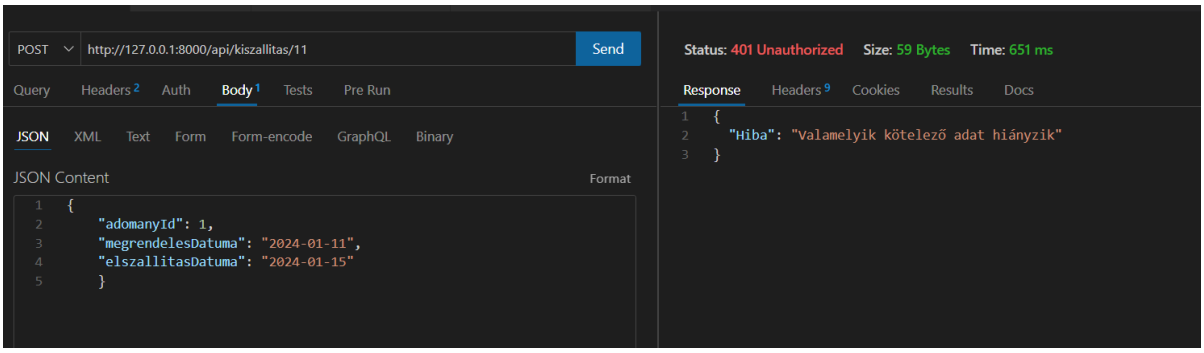


56. Sikeres GetById teszt

b. Az backend sikertelen tesztjeinek képernyőfotói (Thunder Client)

POST:

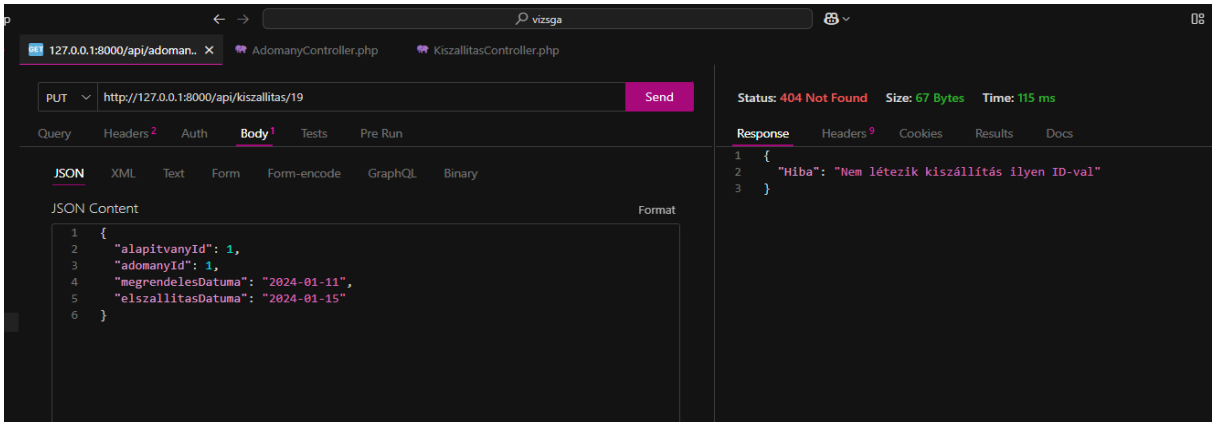
A POST kérés sikertelen lehet, ha a kötelező adatok nem kerülnek elküldésre. Ilyenkor a backend hibát ad vissza, jelezve, hogy hiányos az adatbevitel. Ez segít megelőzni a hibás vagy hiányos adatok mentését az adatbázisba.



57. Sikertelen Post teszt

PUT:

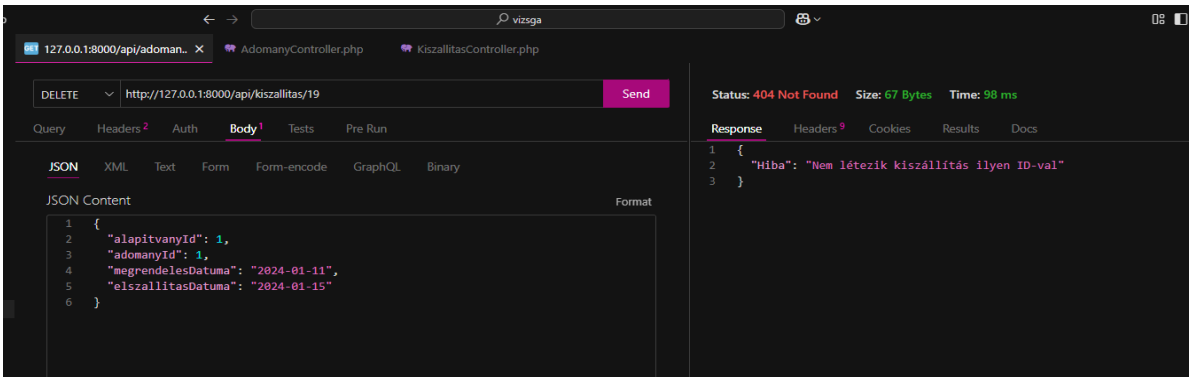
A PUT kérés sikertelen lehet, ha a módosítani kívánt adat nem létezik az adatbázisban. Ebben az esetben a backend hibát küld vissza, amely jelzi, hogy az adott azonosítóhoz nem tartozik rekord. Ez biztosítja, hogy csak létező adatok kerüljenek módosításra.



58. Sikertelen Put teszt

DELETE:

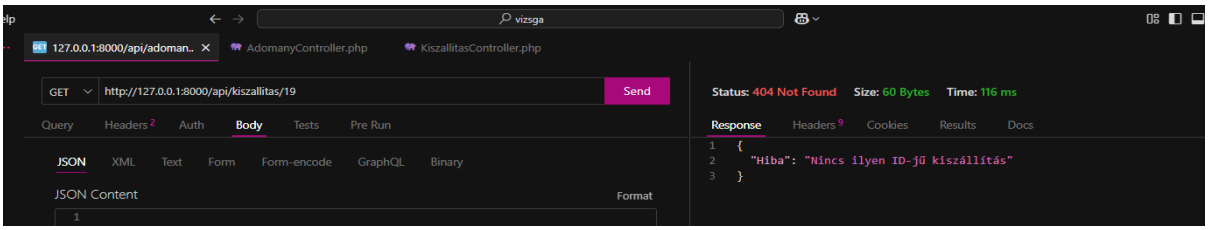
A DELETE kérés akkor lehet sikertelen, ha a törölni kívánt adat nem található az adatbázisban. Ilyenkor a rendszer hibát jelez, és nem történik törlés.



59. Sikertelen Delete teszt

GETBYID:

A GET BY ID kérés sikertelen lehet, ha a megadott azonosítóhoz nem tartozik adat az adatbázisban. Ilyenkor a backend hibát küld vissza, amely jelzi, hogy a keresett elem nem található.



60. Sikertelen GetById teszt

c. Hibakezelés a frontend oldalon

```
uzenet(szoveg: string, tipus: 'success' | 'error' | 'warn' = 'success') {
  this.snackBar.open(szoveg, 'Bezár', {
    duration: 3000,
    horizontalPosition: 'center',
    verticalPosition: 'bottom',
    panelClass: [`${tipus}-snackbar`]
  });
}
```

61. Snackbar üzenetkezelés

Az alkalmazás a felhasználói visszajelzések megjelenítésére egy központi uzenet() metódust használ, amely Angular Material Snackbar segítségével jeleníti meg az értesítéseket. A metódus paraméterként megkapja a megjelenítendő szöveget és az üzenet típusát (például sikeres, hiba vagy figyelmeztetés).

62. Bejelentkezés és regisztráció validációval

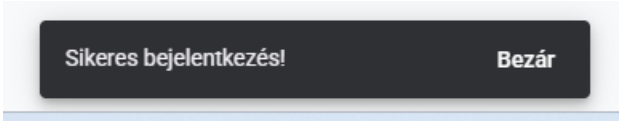
63. Bejelentkezés és regisztráció validációval

A regisztrációs és a bejelentkezős felületen a frontend validáció biztosítja, hogy a felhasználó csak megfelelő adatokat tudjon megadni. Az űrlap Angular template-driven form megoldással készült, ahol minden mezőhöz külön validáció tartozik.

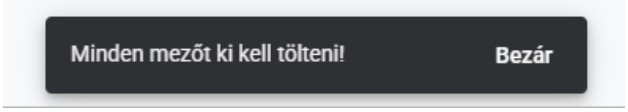
```
<mat-form-field appearance="outline" class="w-100">
  <mat-label>Email</mat-label>
  <input matInput required type="email" name="email" [(ngModel)]="email" #email2Ref="ngModel">
</mat-form-field>

@if(email2Ref.invalid && email2Ref.touched){
<mat-error>Email megadása kötelező</mat-error>
}
```

64. Bejelentkezés és regisztráció validációval

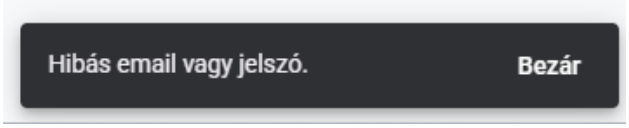
Bejelentkezés és regisztráció hibakezelés üzenetek:



65. Snackbar üzenet: bejelentkezés

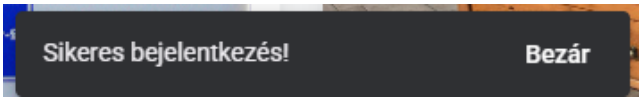


66. Snackbar hibaüzenet: regisztráció

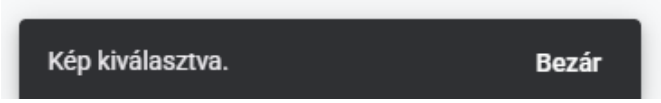


67. Snackbar hibaüzenet: bejelentkezés és regisztráció

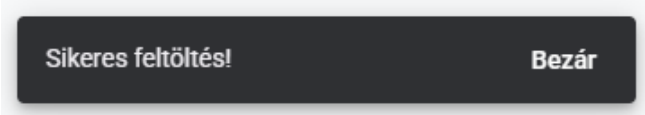
Adományozó oldal hibakezelés üzenetek:



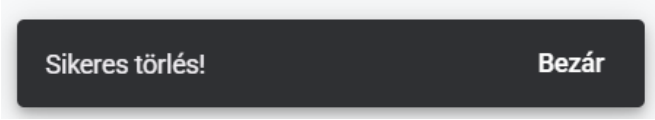
68. Snackbar üzenet: adományozó oldal



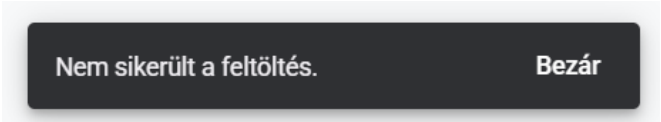
69. Snackbar üzenet: adományozó oldal



70. Snackbar üzenet: adományozó oldal

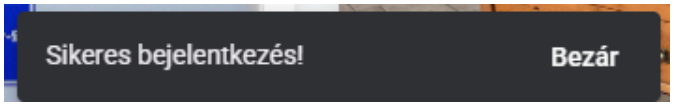


71. Snackbar üzenet: adományozó oldal

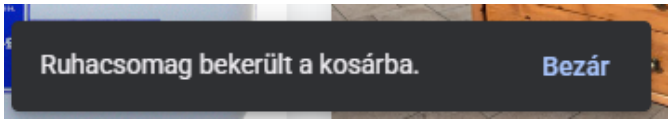


72. Snackbar hibaüzenet: adományozó oldal

Alapítvány hibakezelés üzenetek:

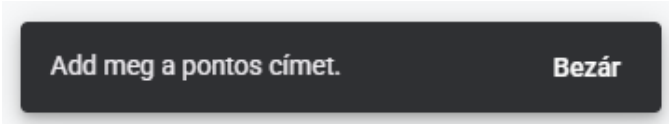


73. Snackbar üzenet: alapítvány oldal

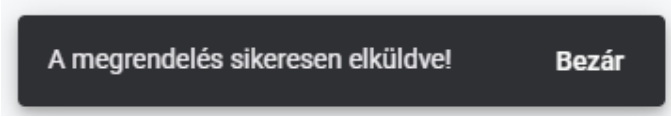


74. Snackbar üzenet: alapítvány oldal

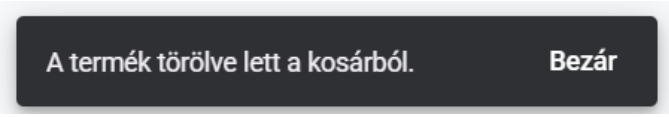
Kosár hibakezelés üzenetek:



75. Snackbar hibaüzenet: kosár oldal

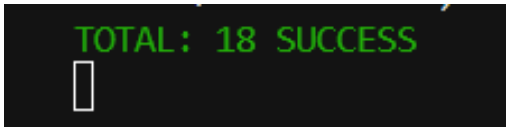


76. Snackbar üzenet: kosár oldal

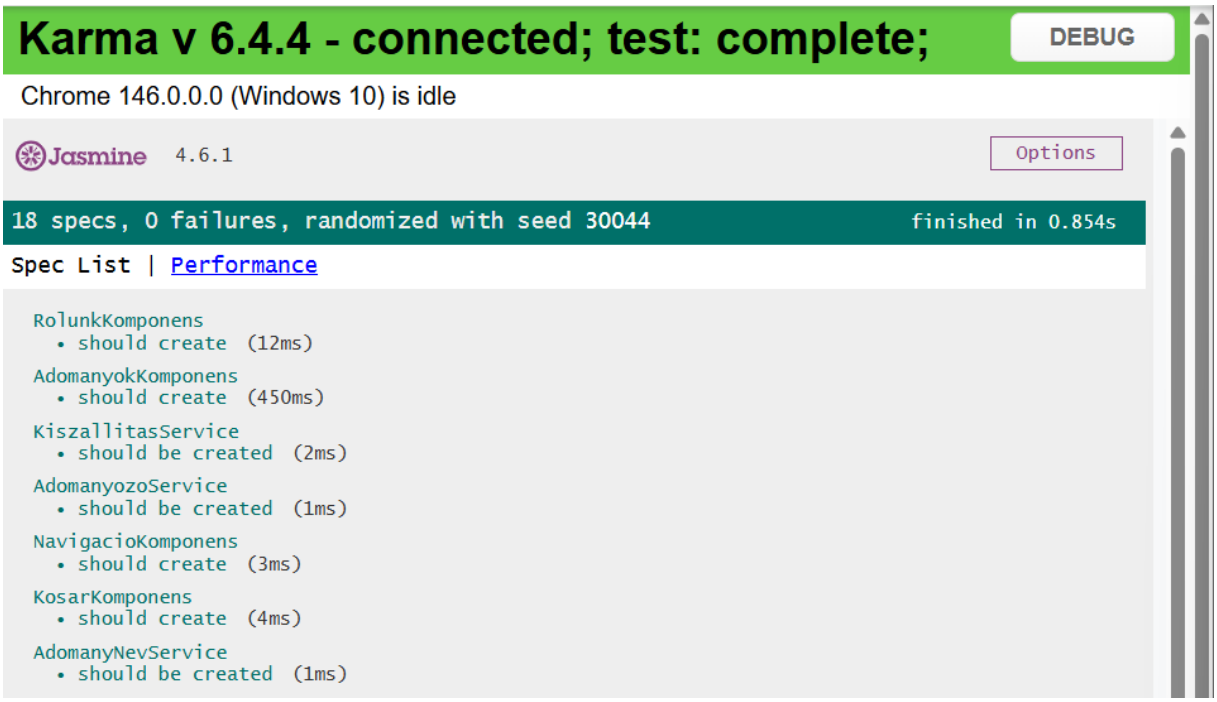


77. Snackbar üzenet: kosár oldal

Jasmine + Karma tesztelés:



78. Jasmine + Karma teszteredmény



79. Jasmine + Karma teszteredmény

Az alkalmazás frontend részéhez egységtesztek készültek, amelyek a komponensek és service-ek alapvető működését ellenőrzik. A tesztek futtatása során minden vizsgált elem sikeresen lefutott, hiba nélkül.

8. Továbbfejlesztési lehetőségek

Mobilalkalmazás fejlesztése a könnyebb használatért

Egy nonprofit adományozó cég működésében nagy előrelépést jelenthet egy mobilalkalmazás fejlesztése. Egy ilyen app lehetővé tenné, hogy a felhasználók gyorsan és egyszerűen adományozzanak, akár néhány kattintással. Emellett értesítéseket is kaphatnának aktuális kampányokról, sürgős segítségkérésekről, vagy arról, hogy az adományuk milyen hatást ért el. Ez nemcsak kényelmesebbé teszi a használatot, hanem erősíti a kapcsolatot is az adományozók és a szervezet között.

AI chatbot a kommunikálás és segítség céljából

Egy AI-alapú chatbot bevezetése nagyban javíthatja a kommunikációt. A chatbot képes lehet válaszolni a gyakran ismételt kérdésekre, segíteni az adományozási folyamatban, vagy eligazítani a felhasználókat a különböző projektek között. Ez különösen hasznos lehet, mert a felhasználók bármikor, azonnal segítséget kaphatnak, anélkül hogy egy valódi ügyfélszolgálatosra kellene várniuk.

9. Irodalomjegyzék

- https://getbootstrap.com/
- https://angular.dev/
- https://material.angular.dev/
- https://laravel.com